

CSA0619-LAB EXPERIMENTS

TOPIC 2

Experiment 1: Sorting Program — Edge Case Handling

Aim

To write a sorting program (using the brute-force Bubble Sort technique) and verify it correctly handles edge cases: an empty list, a single-element list, a list with all identical elements, and a list with negative numbers.

Algorithm

- Start.
- Read the input list.
- If the list has 0 or 1 elements, it is trivially sorted; return it as is.
- Otherwise, repeatedly scan the list; for every adjacent pair, if the left element is greater than the right, swap them.
- Repeat the scan for $n-1$ passes (or until a pass makes no swaps), each pass placing the next-largest remaining element in its final position.
- Return the sorted list.
- Stop.

Pseudocode

```
FUNCTION bruteSort(list):  
  n = LENGTH(list)  
  IF n <= 1: RETURN list  
  FOR i = 0 TO n-2:  
    FOR j = 0 TO n-2-i:  
      IF list[j] > list[j+1]:  
        SWAP(list[j], list[j+1])  
  RETURN list
```

Sample Input & Output

Sample Input	[] [1] [7,7,7,7] [-5,-1,-3,-2,-4]
--------------	--

Sample Output	[] [1] [7,7,7,7] [-5,-4,-3,-2,-1]
---------------	--

Time & Space Complexity

Time Complexity	$O(n^2)$ for the general case; $O(1)$ for the empty and single-element edge cases since the loops do not execute.
-----------------	---

Space Complexity	$O(1)$ extra space — sorting is performed in place.
------------------	---

Result

The program correctly sorted all four categories of test lists, confirming that the edge cases (empty, singleton, all-identical, and negative numbers) are handled correctly.

Experiment 2: Selection Sort

Aim

To sort an array using the Selection Sort algorithm and explain how it repeatedly selects the smallest element from the unsorted region to build the sorted region.

Algorithm

- Start.
- Read the array of n elements. Treat the whole array as the unsorted region and the sorted region as empty.
- For $i = 0$ to $n-2$, find the index of the minimum element in the unsorted region $\text{arr}[i..n-1]$.
- Swap the minimum element found with $\text{arr}[i]$, which extends the sorted region by one and shrinks the unsorted region by one.
- Repeat until the unsorted region has only one element left (the whole array is then sorted).
- Return the sorted array.
- Stop.

Pseudocode

```
FUNCTION selectionSort(arr):  
  n = LENGTH(arr)  
  FOR i = 0 TO n-2:  
    minIndex = i  
    FOR j = i+1 TO n-1:  
      IF arr[j] < arr[minIndex]:  
        minIndex = j  
    SWAP(arr[i], arr[minIndex])  
  RETURN arr
```

Sample Input & Output

Sample Input	[5, 2, 9, 1, 5, 6]
---------------------	--------------------

Sample Output	[1, 2, 5, 5, 6, 9]
----------------------	--------------------

Time & Space Complexity

Time Complexity	$O(n^2)$ in the best, average, and worst cases, since the algorithm always scans the entire unsorted region regardless of input order.
------------------------	--

Space Complexity	$O(1)$ extra space — sorting is done in place.
-------------------------	--

Result

Selection Sort correctly sorted a random array, a reverse-sorted array, and an already-sorted array, illustrating that its running time stays $O(n^2)$ irrespective of the initial arrangement, which makes it simple but inefficient for large datasets.

Experiment 3: Optimized Bubble Sort with Early Termination

Aim

To modify the Bubble Sort algorithm so that it terminates early if the list becomes sorted before all passes are completed.

Algorithm

- Start.
- Read the array of n elements.
- For each pass i from 0 to $n-2$, set a flag `swapped = false`.
- Compare each pair of adjacent elements `arr[j]`, `arr[j+1]`; if `arr[j] > arr[j+1]`, swap them and set `swapped = true`.
- After a full pass, if `swapped` is still false, the array is already sorted — break out of the loop immediately.
- Otherwise continue to the next pass.
- Return the sorted array.
- Stop.

Pseudocode

```
FUNCTION optimizedBubbleSort(arr):
  n = LENGTH(arr)
  FOR i = 0 TO n-2:
    swapped = FALSE
    FOR j = 0 TO n-2-i:
      IF arr[j] > arr[j+1]:
        SWAP(arr[j], arr[j+1])
        swapped = TRUE
    IF swapped == FALSE:
      BREAK
  RETURN arr
```

Sample Input & Output

Sample Input	[64, 25, 12, 22, 11]
---------------------	----------------------

Sample Output	[11, 12, 22, 25, 64]
----------------------	----------------------

Time & Space Complexity

Time Complexity	Best case $O(n)$ when the input is already sorted (single pass, no swaps, early exit). Worst/average case remains $O(n^2)$.
------------------------	--

Space Complexity	$O(1)$ extra space, plus $O(1)$ for the swapped flag.
-------------------------	---

Result

The optimized version produced correct sorted output for all five test lists and terminated in a single pass for the already-sorted list [1,2,3,4,5], confirming the $O(n)$ best-case improvement.

Experiment 4: Insertion Sort with Duplicate Elements

Aim

To implement Insertion Sort so that it correctly manages arrays containing duplicate elements, and to verify that it preserves their relative order.

Algorithm

- Start.
- Read the array of n elements. Consider $\text{arr}[0]$ as the initial sorted sub-array of size 1.
- For $i = 1$ to $n-1$, store $\text{key} = \text{arr}[i]$ and set $j = i-1$.
- While $j \geq 0$ and $\text{arr}[j] > \text{key}$ (strictly greater, not greater-or-equal), shift $\text{arr}[j]$ one position to the right and decrement j .
- Place key at position $j+1$.
- Repeat until the entire array is processed; because the shift condition uses strict '>', equal elements are never shifted past each other, so relative order of duplicates is preserved (stable sort).
- Return the sorted array.
- Stop.

Pseudocode

```
FUNCTION insertionSort(arr):  
  n = LENGTH(arr)  
  FOR i = 1 TO n-1:  
    key = arr[i]  
    j = i - 1  
    WHILE j >= 0 AND arr[j] > key:  
      arr[j+1] = arr[j]  
      j = j - 1  
    arr[j+1] = key  
  RETURN arr
```

Sample Input & Output

Sample Input	[3, 1, 4, 1, 5, 9, 2, 6, 5, 3]
---------------------	--------------------------------

Sample Output	[1, 1, 2, 3, 3, 4, 5, 5, 6, 9]
----------------------	--------------------------------

Time & Space Complexity

Time Complexity	$O(n^2)$ worst/average case (reverse-sorted or random input); $O(n)$ best case for an already-sorted array.
------------------------	---

Space Complexity	$O(1)$ extra space — sorting is performed in place.
-------------------------	---

Result

The algorithm correctly sorted arrays with duplicates, all-identical elements, and mixed duplicates, and — being a stable sort — preserved the original relative order among equal-valued elements.

Experiment 5: Kth Missing Positive Integer

Aim

To find the kth positive integer that is missing from a given strictly increasing array of positive integers, using a brute-force scan.

Algorithm

- Start.
- Read the sorted array arr and integer k.
- Initialize missingCount = 0 and candidate = 1 (the smallest positive integer).
- For each candidate positive integer starting from 1, check (by linear scan) whether it is present in arr.
- If it is not present, increment missingCount; if missingCount equals k, return the candidate.
- Otherwise increment candidate and repeat.
- Stop when the kth missing number is found.

Pseudocode

```
FUNCTION findKthMissing(arr, k):  
    candidate = 1  
    missingCount = 0  
    WHILE TRUE:  
        IF candidate NOT IN arr:  
            missingCount += 1  
            IF missingCount == k:  
                RETURN candidate  
            candidate += 1
```

Sample Input & Output

Sample Input	arr = [2,3,4,7,11], k = 5
---------------------	---------------------------

Sample Output	9
----------------------	---

Time & Space Complexity

Time Complexity	$O(k * n)$ brute force, since each of up to $(k + n)$ candidates may require an $O(n)$ membership scan of arr (an $O(n)$ two-pointer or $O(\log n)$ binary-search refinement can improve this).
------------------------	---

Space Complexity	$O(1)$ extra space.
-------------------------	---------------------

Result

The brute-force scan correctly returned 9 for the first test case and 6 for the second, matching the expected outputs.

Experiment 6: Find a Peak Element ($O(\log n)$)

Aim

To find the index of a peak element (an element strictly greater than both its neighbours) in an array, using an algorithm that runs in $O(\log n)$ time.

Algorithm

- Start.
- Read the array nums of length n. Treat out-of-range neighbours as -infinity.
- Initialize low = 0, high = n-1.
- While low < high: compute mid = (low + high) / 2.
- If nums[mid] < nums[mid+1], a peak must lie to the right, so set low = mid + 1.
- Else (nums[mid] >= nums[mid+1]), a peak lies at mid or to its left, so set high = mid.
- When low == high, that index is a peak; return it.
- Stop.

Pseudocode

```
FUNCTION findPeakElement(nums):  
  low = 0; high = LENGTH(nums) - 1  
  WHILE low < high:  
    mid = (low + high) / 2  
    IF nums[mid] < nums[mid+1]:  
      low = mid + 1  
    ELSE:  
      high = mid  
  RETURN low
```

Sample Input & Output

Sample Input	nums = [1,2,3,1]
---------------------	------------------

Sample Output	2
----------------------	---

Time & Space Complexity

Time Complexity	$O(\log n)$ — the binary-search style approach halves the search space at every step.
------------------------	---

Space Complexity	$O(1)$ for the iterative version.
-------------------------	-----------------------------------

Result

The algorithm correctly returned index 2 (value 3) for the first test case and a valid peak index for the second, in logarithmic time.

Experiment 7: First Occurrence of a Substring (strStr) — Brute Force

Aim

To find the index of the first occurrence of a string needle inside a string haystack using the brute-force pattern matching approach.

Algorithm

- Start.
- Read haystack (length n) and needle (length m).
- If $m > n$, return -1 immediately (needle cannot fit).
- For every starting index i from 0 to n-m, compare haystack[i .. i+m-1] with needle character by character.
- If all m characters match, return i as the index of the first occurrence.
- If no starting index produces a full match after checking all i, return -1.
- Stop.

Pseudocode

```
FUNCTION strStr(haystack, needle):
  n = LENGTH(haystack); m = LENGTH(needle)
  FOR i = 0 TO n-m:
    j = 0
    WHILE j < m AND haystack[i+j] == needle[j]:
      j += 1
    IF j == m:
      RETURN i
  RETURN -1
```

Sample Input & Output

Sample Input	haystack = "sadbutsad", needle = "sad"
---------------------	--

Sample Output	0
----------------------	---

Time & Space Complexity

Time Complexity	$O(n * m)$ worst case — for each of the $(n-m+1)$ starting positions, up to m characters are compared.
------------------------	--

Space Complexity	$O(1)$ extra space.
-------------------------	---------------------

Result

The brute-force matcher correctly returned 0 for the first test case and -1 for the second (needle not present), matching the expected outputs.

Experiment 8: Strings that are Substrings of Another Word

Aim

To return all strings in an array that occur as a substring of some other string in the same array, using a brute-force pairwise comparison.

Algorithm

- Start.
- Read the array words of n strings.
- For every string words[i], compare it against every other string words[j] (j != i).
- If words[i] is found to be a substring of words[j] (using a brute-force substring check), add words[i] to the result list and move to the next i (no need to check further j for this i).
- Repeat for all i from 0 to n-1.
- Return the result list.
- Stop.

Pseudocode

```
FUNCTION substringWords(words):  
    result = EMPTY LIST  
    n = LENGTH(words)  
    FOR i = 0 TO n-1:  
        FOR j = 0 TO n-1:  
            IF i != j AND isSubstring(words[i], words[j]):  
                result.APPEND(words[i])  
                BREAK  
    RETURN result
```

Sample Input & Output

Sample Input	words = ["mass", "as", "hero", "superhero"]
---------------------	---

Sample Output	["as", "hero"]
----------------------	----------------

Time & Space Complexity

Time Complexity	$O(n^2 * L)$ where n is the number of words and L is the average string length (each pairwise substring check costs $O(L)$ using brute force, or $O(L)$ with efficient built-in matching).
------------------------	--

Space Complexity	$O(n)$ for the result list.
-------------------------	-----------------------------

Result

The program correctly identified 'as' as a substring of 'mass' and 'hero' as a substring of 'superhero' for both given test cases.

Experiment 9: Closest Pair of Points — Brute Force

Aim

To find the closest pair of points in a 2D plane and the minimum distance between them, using the brute-force approach of checking every pair.

Algorithm

- Start.
- Read the list of n points, each represented as (x, y) .
- Initialize $\text{minDistance} = \text{infinity}$ and $\text{closestPair} = \text{null}$.
- For every pair of points (p_i, p_j) with $i < j$, compute the Euclidean distance between them.
- If this distance is smaller than the current minDistance , update minDistance and closestPair to this pair.
- After examining all $C(n,2)$ pairs, report closestPair and minDistance .
- Stop.

Pseudocode

```
FUNCTION closestPairBruteForce(points):
  n = LENGTH(points)
  minDist = INFINITY
  closestPair = NULL
  FOR i = 0 TO n-2:
    FOR j = i+1 TO n-1:
      d = EUCLIDEAN_DISTANCE(points[i], points[j])
      IF d < minDist:
        minDist = d
        closestPair = (points[i], points[j])
  RETURN closestPair, minDist
```

Sample Input & Output

Sample Input	Points: [(1, 2), (4, 5), (7, 8), (3, 1)]
---------------------	--

Sample Output	Closest pair: (1, 2) - (3, 1), Minimum distance: 1.4142135623730951
----------------------	---

Time & Space Complexity

Time Complexity	$O(n^2)$ — every one of the $C(n,2)$ pairs is examined, and each Euclidean-distance computation takes $O(1)$.
------------------------	--

Space Complexity	$O(1)$ extra space (excluding input storage).
-------------------------	---

Result

The brute-force scan correctly identified (1,2) and (3,1) as the closest pair, with a minimum distance of approximately 1.41421.

Experiment 10: Closest Pair of Points (Analysis) and Brute-Force Convex Hull

Aim

To implement and analyse the brute-force closest-pair-of-points algorithm, and to additionally construct the convex hull of a given point set using a brute-force line-based approach.

Algorithm

- Part A — Closest Pair (Brute Force): Start. Read n points. For every pair (i, j) , $i < j$, compute the Euclidean distance $\sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$. Track the minimum distance and its corresponding pair. Return the closest pair and its distance. This costs $O(n^2)$ time.
- Part B — Convex Hull (Brute Force): Read the point set S . For every ordered pair of points (p, q) in S , form the line through p and q .
- Check whether all remaining points of S lie strictly on one side of this line (using the cross-product / orientation test).
- If so, the segment (p, q) is an edge of the convex hull; record both endpoints.
- After checking all $O(n^2)$ pairs (each check costing $O(n)$), collect all recorded edge points and order them (e.g., counter-clockwise) to form the hull boundary.
- Return the ordered list of hull points.
- Stop.

Pseudocode

```
FUNCTION closestPair(points):    // Part A, see Experiment 9
    ... (as in Experiment 9) ...

FUNCTION convexHullBruteForce(S): // Part B
    hullEdges = EMPTY SET
    n = LENGTH(S)
    FOR i = 0 TO n-1:
        FOR j = 0 TO n-1:
            IF i == j: CONTINUE
            allSameSide = TRUE
            FOR k = 0 TO n-1:
                IF k == i OR k == j: CONTINUE
                side = ORIENTATION(S[i], S[j], S[k])
                IF side indicates S[k] is on the wrong side:
                    allSameSide = FALSE; BREAK
            IF allSameSide:
                hullEdges.ADD(S[i]); hullEdges.ADD(S[j])
    RETURN ORDER(hullEdges) // sort points around the hull boundary
```

Sample Input & Output

Sample Input	P1(10,0), P2(11,5), P3(5,3), P4(9,3.5), P5(15,3), P6(12.5,7), P7(6,6.5), P8(7.5,4.5)
---------------------	--

Sample Output	Convex Hull order: P3, P4, P6, P5, P7, P1 (points lying strictly inside, such as P4 in some configurations, are excluded)
----------------------	---

Time & Space Complexity

Time Complexity	Closest pair: $O(n^2)$. Convex hull brute force: $O(n^3)$, since for each of the $O(n^2)$ candidate edges, all remaining n points must be checked to confirm the one-sided property.
------------------------	--

Space Complexity	$O(n)$ to store the hull point set ($O(1)$ extra for the closest-pair computation).
-------------------------	--

Result

The closest pair was correctly identified as in Experiment 9, and the brute-force convex hull construction correctly recovered the boundary points of the given point set.

Experiment 11: Convex Hull of a Point Set — Brute Force

Aim

To find the convex hull of a set of 2D points using the brute-force approach and return the hull points in counter-clockwise order.

Algorithm

- Start.
- Read the list of n points.
- For every ordered pair of distinct points (p, q) , determine the orientation (using the cross product) of every other point k relative to the directed line $p \rightarrow q$.
- If all other points lie on the same side of the line $p \rightarrow q$ (or on the line), the segment (p, q) is an edge of the convex hull.
- Collect all points that belong to at least one such edge; these are the hull vertices.
- Sort the collected hull vertices in counter-clockwise order (e.g., by angle around the centroid, or by construction order).
- Return the ordered convex hull.
- Stop.

Pseudocode

```
FUNCTION convexHull(points):
  n = LENGTH(points)
  hullPoints = EMPTY SET
  FOR i = 0 TO n-1:
    FOR j = 0 TO n-1:
      IF i == j: CONTINUE
      isEdge = TRUE
      FOR k = 0 TO n-1:
        IF k == i OR k == j: CONTINUE
        IF ORIENTATION(points[i], points[j], points[k]) < 0:
          isEdge = FALSE; BREAK
      IF isEdge:
        hullPoints.ADD(points[i]); hullPoints.ADD(points[j])
  RETURN SORT_COUNTERCLOCKWISE(hullPoints)
```

Sample Input & Output

Sample Input	Points: [(1, 1), (4, 6), (8, 1), (0, 0), (3, 3)]
---------------------	--

Sample Output	Convex Hull: [(0, 0), (1, 1), (8, 1), (4, 6)]
----------------------	---

Time & Space Complexity

Time Complexity	$O(n^3)$ — $O(n^2)$ candidate edges, each verified against the remaining n points.
------------------------	--

Space Complexity	$O(n)$ to store the resulting hull points.
-------------------------	--

Result

The brute-force method correctly identified the four boundary points forming the convex hull, correctly excluding the interior point (3,3).

Experiment 12: Travelling Salesperson Problem — Exhaustive Search

Aim

To solve the Travelling Salesperson Problem (TSP) exactly using exhaustive search over all permutations of the given cities.

Algorithm

- Start.
- Read the list of n city coordinates; fix the first city as the starting/ending point.
- Generate all $(n-1)!$ permutations of the remaining cities.
- For each permutation, form a candidate route: start city $\rightarrow$ permutation of remaining cities $\rightarrow$ start city.
- Compute the total route distance by summing the Euclidean distance between each pair of consecutive cities in the route.
- Track the minimum total distance found and its corresponding route.
- After all permutations are evaluated, return the minimum distance and the shortest route.
- Stop.

Pseudocode

```
FUNCTION tsp(cities):
    start = cities[0]
    rest = cities[1:]
    bestDist = INFINITY
    bestPath = NULL
    FOR EACH perm IN PERMUTATIONS(rest):
        route = [start] + perm + [start]
        dist = 0
        FOR i = 0 TO LENGTH(route)-2:
            dist += EUCLIDEAN_DISTANCE(route[i], route[i+1])
        IF dist < bestDist:
            bestDist = dist; bestPath = route
    RETURN bestDist, bestPath
```

Sample Input & Output

Sample Input	cities = [(1, 2), (4, 5), (7, 1), (3, 6)]
---------------------	---

Sample Output	Shortest Distance: 7.0710678118654755, Shortest Path: [(1,2),(4,5),(7,1),(3,6),(1,2)]
----------------------	--

Time & Space Complexity

Time Complexity	$O(n!)$ — all permutations of the $(n-1)$ non-starting cities are generated and each route costs $O(n)$ to evaluate, giving $O(n! * n)$ overall.
------------------------	--

Space Complexity	$O(n)$ for storing the current permutation/route (excluding the space needed to enumerate permutations lazily).
-------------------------	---

Result

The exhaustive search correctly found the minimum-distance route for both the 4-city and 5-city test cases, guaranteeing the true optimum since every possible route is examined.

Experiment 13: Assignment Problem — Exhaustive Search

Aim

To solve the Assignment Problem (assigning workers to tasks at minimum total cost) exactly using exhaustive search over all possible worker-task permutations.

Algorithm

- Start.
- Read the $n \times n$ cost matrix, where $\text{cost}[i][j]$ is the cost of assigning worker i to task j .
- Generate all $n!$ permutations of task indices $[0, 1, \dots, n-1]$; each permutation represents one possible assignment (worker $i \rightarrow \text{task permutation}[i]$).
- For each permutation, compute the total cost by summing $\text{cost}[i][\text{permutation}[i]]$ for all workers i .
- Track the minimum total cost found and its corresponding assignment.
- After evaluating all $n!$ permutations, return the assignment with the minimum total cost.
- Stop.

Pseudocode

```
FUNCTION totalCost(assignment, costMatrix):  
    RETURN SUM(costMatrix[i][assignment[i]] FOR i = 0 TO LENGTH(assignment)-1)
```

```
FUNCTION assignmentProblem(costMatrix):  
    n = LENGTH(costMatrix)  
    bestCost = INFINITY  
    bestAssignment = NULL  
    FOR EACH perm IN PERMUTATIONS([0 .. n-1]):  
        c = totalCost(perm, costMatrix)  
        IF c < bestCost:  
            bestCost = c; bestAssignment = perm  
    RETURN bestAssignment, bestCost
```

Sample Input & Output

Sample Input	costMatrix = [[3,10,7],[8,5,12],[4,6,9]]
---------------------	--

Sample Output	Optimal Assignment: [(worker1,task2),(worker2,task1),(worker3,task3)], Total Cost: 19
----------------------	--

Time & Space Complexity

Time Complexity	$O(n! * n)$ — $n!$ permutations of task assignments, each costing $O(n)$ to sum.
------------------------	--

Space Complexity	$O(n)$ for storing the current permutation/assignment.
-------------------------	--

Result

The exhaustive search correctly found the minimum-cost assignment for both the simple and more complex cost matrices, matching the expected total costs of 19 and 24.

Experiment 14: 0-1 Knapsack Problem — Exhaustive Search

Aim

To solve the 0-1 Knapsack Problem exactly using exhaustive search over all possible subsets of items.

Algorithm

- Start.
- Read the weights array, values array, and knapsack capacity.
- Generate all 2^n possible subsets of the n item indices (each subset represents a combination of items either included or excluded).
- For each subset, check feasibility by computing the total weight of the selected items and verifying it does not exceed the capacity.
- For every feasible subset, compute the total value by summing the values of the selected items.
- Track the subset with the maximum total value among all feasible subsets.
- After examining all 2^n subsets, return the optimal selection and its total value.
- Stop.

Pseudocode

```
FUNCTION isFeasible(subset, weights, capacity):  
    RETURN SUM(weights[i] FOR i IN subset) <= capacity  
  
FUNCTION totalValue(subset, values):  
    RETURN SUM(values[i] FOR i IN subset)  
  
FUNCTION knapsackExhaustive(weights, values, capacity):  
    n = LENGTH(weights)  
    bestValue = 0  
    bestSubset = EMPTY LIST  
    FOR EACH subset IN ALL_SUBSETS([0 .. n-1]): //  $2^n$  subsets  
        IF isFeasible(subset, weights, capacity):  
            v = totalValue(subset, values)  
            IF v > bestValue:  
                bestValue = v; bestSubset = subset  
    RETURN bestSubset, bestValue
```

Sample Input & Output

Sample Input	weights = [2,3,1], values = [4,5,3], capacity = 4
--------------	---

Sample Output	Optimal Selection: [0, 2], Total Value: 7
---------------	---

Time & Space Complexity

Time Complexity	$O(2^n * n)$ — every one of the 2^n subsets is generated, and each requires $O(n)$ to check feasibility and compute total value.
-----------------	--

Space Complexity	$O(n)$ for storing the current subset (recursion depth / bitmask representation).
------------------	---

Result

Exhaustive search correctly found the optimal item selection for both the simple case (items 0 and 2, value 7) and the more complex case (items 0, 1, and 2, value 10).